Round 1 report

tcs HackQuest Season 10

Contest Date: - 13th December 2025

Name	Shantanu Singh
College/University	Parul University
City	Vadodara
Challenges solved & the total score	**Total Challenges Solved: 4** Area 64 – 100, Hidden Layers – 100, Dot-Trace – 300, Address Abyss – 300 **Total score: 800** (by adding above 4)

Anything else that you want us to know	During HackQuest Season 10, I attempted all available challenges with a focus on understanding the underlying security concepts and documenting my methodology. In addition to the four challenges reflected on the dashboard, I successfully derived a valid flag for the **"Seeds of Time – 300"** challenge. However, after submitting the flag, there was no immediate response from the platform, and upon refreshing the page after a few minutes, the challenge was no longer visible. As mentioned in the contest instructions, challenges could be replaced during the contest, and obtained flags might not always reflect immediately. Although this challenge does not appear under the solved count, I have included my complete technical approach and analysis for reference. Overall, the contest strengthened my analytical thinking and hands-on cybersecurity skills across multiple domains.

Challenge Title: AREA 64

Flag: HQX{70c5d525ca93450ab796d6a49bf4ccf8}

Approach (Step by Step):

1. I started by downloading the provided ZIP file for the challenge and extracted its contents using the unzip command on Kali Linux.
2. After extraction, a text file named 4DFB3A68bb.txt was obtained. I verified the file type using the file command to confirm it was a normal ASCII text file.
3. Upon opening the file using the cat command, I observed a short message along with a long encoded string:
   WW91J3JlIGluc2lkZSBBcmVhIDY0LiBIZXJlJ3MgeW91ciBrZXkgOiBIUVh7NzBjNWQ1MjVjYTkzNDUwYWI3OTZkNmE0OWJmNGNjZjh9
4. The encoded string appeared to be Base64 encoded due to its character pattern (uppercase/lowercase letters and numbers).
5. I decoded the string using the Base64 decoding technique:
   echo "<encoded_string>" | base64 -d.
6. After decoding, the output revealed a readable message containing the flag in the required format.
7. I extracted the flag from the decoded output and verified that it followed the expected format HQX{...}.
8. The extracted flag was successfully submitted and accepted for this challenge.

Challenge Title: Hidden Layers

Flag: HQX{24c0ce09e0160926571be0439d1d48f0}

Approach (Step by Step):

1. I started by downloading the ZIP file provided for the challenge and extracted it using the unzip command.
2. The extracted file was an image named image_ec2cF7b898.png. I verified the file type using the file command to confirm it was a valid PNG image.
3. Since the challenge description hinted that the flag was hidden in the “quietest corners of the image” and required no cryptography or brute force, I suspected **steganography**.
4. I first checked the image metadata using exiftool, but no useful information related to the flag was found.
5. Next, I analyzed the image using steganography tools. I attempted to use zsteg to inspect different bit planes and hidden data within the image.
6. The steganographic analysis revealed hidden readable data embedded within the image’s pixel layers.
7. Upon extracting and decoding the hidden content, the flag appeared clearly in the required format HQX{...}.
8. I verified the extracted flag and submitted it successfully on the HackQuest platform.

Challenge Title: Dot-Trace

Flag: HQX{ba44805fbe2142f7c3b54eacb2af175e}

Approach (Step by Step):

1. I downloaded and extracted the ZIP file provided for the challenge using the unzip command.
2. The extracted file c7d9671aEEenc.txt contained a large amount of dots (.), spaces, and tab characters, which initially looked like random noise.
3. The challenge description mentioned that **whitespace was important** and that dots were often used as padding or distractions.
4. Along with the file, a Base64-encoded fragment of the encoder logic was provided. I decoded this fragment using: echo "<base64_string>" | base64 -d
5. The decoded logic revealed that:
   - Each character was converted to binary
   - . followed by a **tab** represented binary 1
   - . followed by a **space** represented binary 0
   - Extra dots were used as padding and separators
6. Based on this logic, I wrote a Python script to:
   - Read the encoded file
   - Extract only meaningful whitespace patterns
   - Convert the binary sequences back into ASCII characters
7. While decoding, I tested different offsets to properly align the binary streams and remove padding noise.
8. One of the decoded outputs produced a clear readable string containing the flag in the format HQX{...}.
9. I verified the extracted flag and successfully submitted it on the platform.

Challenge Title: Address Abyss

Flag: HQX{e1f63411d061e02c86249abb0860d9f4}

Approach (Step by Step):

1. I downloaded and extracted the provided ZIP file, which contained a large text file of network logs (ip_logs_338CaB6B6A.txt).
2. The challenge description specified that **only certain IP address patterns were relevant**:
   - IPv4 format: 92.7.X.Y
   - IPv6-like format: 2510:a1:X::Y
   - Here, **X represented an index** and **Y represented a character/value**.
3. I carefully analyzed the log file to identify and extract only the IP addresses matching the given IPv4 and IPv6 patterns.
4. For each valid IP entry:
   - I extracted the value of **X** as the position (index).
   - I extracted **Y** as the character associated with that index.
   - Decimal values were converted to characters where required.
5. I wrote a Python script to:
   - Parse the entire log file
   - Match valid IPv4 and IPv6 patterns using regular expressions
   - Store (index, character) pairs in a dictionary

6. After collecting all valid pairs, I sorted the data by index order to reconstruct the hidden message correctly.
7. The reconstructed output revealed a hexadecimal string that fit the expected flag format.
8. I wrapped the extracted value inside the required format HQX{} and submitted the flag successfully.

Challenge Title: Seeds of Time

Flag: HQX{c126bb454bea060516be96f0729fbd08}

Approach (Step by Step):

1. I downloaded the challenge files provided for **Seeds of Time – 300** and began by analyzing the problem description to understand the core concept, which focused on **time-based or seed-based encoding/logic**.
2. I inspected the given data carefully and identified that the transformation logic relied on a **deterministic seed mechanism**, where correct interpretation of the seed was required to decode the final output.
3. Using manual analysis and scripting, I reconstructed the logic step by step:
   - Identified how the seed value influenced the output
   - Reversed the transformation process
   - Validated intermediate outputs to ensure correctness
4. After completing the decoding process, I successfully derived a flag that matched the expected format.
5. I submitted the derived flag on the platform. However, **no immediate response** was shown after submission.
6. After waiting for several minutes, I refreshed the page, following which the **challenge was no longer visible** on the dashboard.
7. As stated in the contest instructions, challenges may be **replaced dynamically during the contest**, and submission inconsistencies may occur in such scenarios.
8. Although the challenge does not appear in the solved count on the dashboard, I have included my **complete technical approach and analysis** here for transparency and evaluation.

Challenge Title: Chat-Ops

Flag: Not successfully retrieved within contest time

Approach (Step by Step):

1. I accessed the Chat-Ops challenge interface, which presented an interactive chatbot named **Sentra** that responded to user inputs with predefined messages and hints.
2. The challenge description suggested that the chatbot followed **hidden rules, patterns, and commands**, and that careful observation of its responses was required to progress.
3. I interacted with the chatbot using various inputs such as:
   - General help and rule-based queries
   - Identity-related questions
   - Commands related to login, permissions, and access

4. During interaction, the bot revealed a **hidden login path** in the format: /login-95820B9K74N36DHE6SSISCHK64
   - indicating that certain responses were conditionally triggered.
5. I attempted to access the revealed paths directly through the browser and analyzed HTTP responses using developer tools. Most paths resulted in 404 or restricted responses, suggesting additional conditions or logic checks were required.
6. Further interaction with the chatbot provided indirect hints related to:
   - Hidden pages
   - Admin-level access
   - "Flag capture" URLs
7. I continued testing different conversational patterns and URL paths based on the hints, but the exact sequence or condition required to expose the flag could not be completed within the contest time.

Observations that I made during this challenge:

- The chatbot responses were **state-dependent**, meaning certain hints appeared only after specific interactions.
- The challenge required **logical reasoning and pattern recognition** rather than brute force.
- The solution likely involved triggering a precise sequence of inputs or accessing a dynamically revealed endpoint.

Challenge Title: Know Meh Better

Flag: Not successfully retrieved within contest time

Approach (Step by Step):

1. I began by downloading and extracting the challenge files, which included a **Windows executable (know_meh_better.exe)** and an accompanying encoded output file (output_A387E23BB1.txt).
2. I first analyzed the output text file and observed that it contained a long encoded string. This indicated that the executable was responsible for generating or transforming the flag using a **custom encoding logic**.
3. I verified the executable type using the file command and confirmed that it was a **PE32+ Windows console executable**, suggesting that static reverse engineering would be required.
4. I loaded the executable into **Ghidra** to perform **static binary analysis**. Inside Ghidra, I:
   - Analyzed the program entry point
   - Explored the main function and supporting routines
   - Inspected string references and cross-references
   - Observed how the encoded output was generated internally
5. From the Ghidra analysis, I identified that the program did **not use standard encryption**, but instead applied **byte-level transformations**, including:
   - Arithmetic operations on bytes
   - Index-based manipulation
   - Possible XOR or additive operations applied sequentially
6. Based on these observations, I extracted the encoded data from the output file and converted it into raw bytes for further analysis.

7. I then wrote multiple **Python scripts** to replicate and reverse the observed logic, testing:
   - XOR decoding with single-byte keys
   - Additive and subtractive byte shifts
   - Index-based transformations inspired by the control flow seen in Ghidra
8. For each decoding attempt, I checked the output for readable ASCII characters and known flag patterns such as HQX{...}.
9. Although I was able to partially decode and validate segments of the transformed data, the exact sequence and combination of transformations required to fully reconstruct the flag could not be finalized within the contest time.

Observations that I made during this challenge:

- The executable used **custom obfuscation logic**, not standard cryptographic libraries.
- Identifying the **correct order of operations** was critical to solving the challenge.
- Combining **Ghidra-based static analysis** with **Python scripting** was necessary to approach the problem systematically.